



BIRC 商智中心

Business Intelligence Center — 推動校園數位人才培育

2026 後端群體驗營

```
$ curl http://localhost:8080/api/products
```

Backend 101

// 網路怎麼跑 · 狀態碼 · Port · 框架的魔法

看得見的請求

TCP/IP、HTTP、Port

讀得懂的回應

200 / 400 / 409 / 500

框架的真相

它替你省了什麼



今天會講什麼

系列規劃與介紹

01

我們在哪裡、要往哪裡走

2 min

常見網站應用

02

你每天都在用的東西

2 min

五大面向拆解網站應用

03

一個網站到底由什麼組成

2 min

網路是如何運作的

04

TCP/IP 四層 · 應用層協議 · HTTP

3 min



先回顧一下：Day 2 我們做到了什麼



Day 2 的成果

- ✓ 用陣列存商品資料
- ✓ 用 JDBC 連 SQLite 資料庫
- ✓ 用 `HttpServer` 開了一個 API
- ✓ 用 `fetch()` 讓瀏覽器呼叫它

? 你可能答不出來的問題

- • 瀏覽器和 Java 是「怎麼」連上的？
- • `8080` 這個數字代表什麼？
- • 為什麼有的請求回 200、有的回 404？
- • 我寫的那 189 行，有人在幫我做事嗎？



今天就是來回答這四個問題的。



常見網站應用 — 你每天都在用



foodpanda

瀏覽餐廳 → 購物車 → 下單

背後：菜單 API、庫存 API、訂單 API



YouTube

看影片、留言、推薦

背後：影片 API、推薦演算法、CDN



網路銀行

查餘額、轉帳、對帳

背後：驗證、加密、稽核日誌



LINE

傳訊息、語音、通知

背後：長連線、推播、訊息佇列



蝦皮

搜尋商品、下單、付款

背後：搜尋引擎、金流、推薦



宅配

查包裹、通知送貨

背後：物流 API、Webhook

五大面向拆解網站應用



前端 Frontend — 使用者看到的

HTML 結構、CSS 樣式、JavaScript 互動。跑在**使用者的瀏覽器**裡，伺服器只送程式碼。



後端 Backend — 使用者看不到的

收請求、驗證、算錢、動資料。跑在**伺服器**裡，你看不到但它一直在運作。



資料庫 Database — 記憶體之外的記憶

MySQL、PostgreSQL、SQLite、MongoDB。程式關掉、機器重開，資料還在。



網路 Network — 把它們連起來

TCP/IP、HTTP、DNS。瀏覽器要能找到伺服器、要能問它要資料。



你每天用的網站，拆開長這樣



瀏覽器

顯示畫面
送出請求



後端伺服器

收請求
執行邏輯



資料庫

保存商品
使用者資料



在中間那三個雙向箭頭上，發生的事，



網路是如何運作的 — 從瀏覽器到伺服器

1. 輸入網址

你在瀏覽器打 `lucashsu.dev`，按 Enter

2. DNS 查詢

問 DNS 伺服器：「`lucashsu.dev` 的 IP 是多少？」 → `140.82.121.4`

3. 連線

TCP 三次握手，建立到 `140.82.121.4:443` 的連線

4. 送請求

送出 HTTP 請求：`GET /index.html HTTP/1.1`

5. 收回應

伺服器回 `200 OK` + HTML，瀏覽器畫出來



DNS 負責「名字 → IP」，HTTP 負責「問什麼、要什麼」。兩件事，別搞混。



TCP/IP 四層模型

4. 應用層 Application 你寫的程序在這層

HTTP、HTTPS、DNS、SMTP、FTP、SSH — 定義「訊息長什麼樣、怎麼問怎麼答」

👉 今天上課的每一行 Java 程式碼都在這裡

3. 傳輸層 Transport 用 Port 號分流

TCP (可靠、慢、要握手)、UDP (不可靠、快)。80、443、3306、8080 都是這層的門牌

👉 第 6 點講的 Port 號在這裡

2. 網際網路層 Internet

IP 協定。負責「把封包送到全世界的正確地址」。140.82.121.4 就是 IP

1. 網路存取層 Network Access



常見的應用層協議

HTTP 80

網頁。request / response，狀態碼，無狀態

HTTPS 443

加密的 HTTP。瀏覽器網址列的鎖就是它

DNS 53

名字查 IP。名字給人看，IP 給機器用

SSH 22

遠端登入伺服器。你連上主機就是走它

FTP / SFTP 21 / 22

傳檔案。早期上傳網站必用

SMTP / IMAP 25 / 143

寄信 / 收信。Gmail 背後就是這兩個

MySQL 3306

資料庫。嚴格來說不算 HTTP 那一類，但同樣是「應用層協議」

Redis 6379

快取。Day 2 我們完全沒用到它

HTTP 到底長什麼樣

 請求 Request (前端 → 後端)

```
1 GET /api/products HTTP/1.1
2 Host: localhost:8080
3 Accept: application/json
```

 回應 Response (後端 → 前端)

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 252
```

第一行叫**起始行**：方法 + 路徑 + 版本

第一行先寫版本，再寫**狀態碼** + 原因片語

我們昨天寫的那行程式碼，就是在做這件事：

`server.createContext("/api/products", ...)` ← 對應 URL 路徑

`exchange.getRequestMethod()` ← 對應 GET / POST

`sendJson(exchange, 200, ...)` ← 對應狀態碼 + body

✓ 檢查點 1 — 網路與 HTTP

✓ 我能說出

- • DNS 做的事
- • TCP/IP 四層由下往上是什麼
- • HTTP 請求有哪幾個部分
- • 「無狀態」的意思

? 我還不太確定

- • TCP 和 UDP 差在哪
- • 為什麼要有四層，不會太多嗎
- • 三次握手在幹嘛

💡 馬上複習

回到「TCP/IP 四層模型」，用「線、路、包、話」重念一次。

1 2
3 4

HTTP 狀態碼 — 2xx 成功

200 OK — 成功，東西在這裡

最常見。GET 成功、POST 成功都回這個。GET /api/products 回的就是 200

前端怎麼處理：`response.json()` 直接用

201 Created — 成功，而且「建立了東西」

跟 200 的差別：201 表示伺服器新建立了一筆資源。例如註冊新帳號、新增一筆訂單

很多實務上會混用 200 代替 201，但嚴格來說「新增」該用 201



2xx 的共同點：伺服器成功處理了你的請求，而且沒有出錯。前端可以放心把資料當成有效值用。

🚫 HTTP 狀態碼 — 4xx 是「你的問題」

400 Bad Request — 你的請求格式不對

送出 `productId=abc`（應該是數字）。伺服器看不懂就拒絕。

我們的程式：`NumberFormatException` → 400 ✓

401 Unauthorized — 你「是誰」我還不知道

沒登入、token 過期、根本沒帶 token。認不出你是誰

前端：導向登入頁面

403 Forbidden — 我知道你是誰，但你不可以

已登入了，但這筆訂單不是你的。401 和 403 的差別就在這裡

前端：顯示「沒有權限」，不要登出

💥 HTTP 狀態碼 — 5xx 是「我的問題」

500 Internal Server Error — 我壞掉了

伺服器裡發生了你預期外的錯誤：程式 exception、資料庫連不上、寫炸了

我們的程式：`catch (Exception) → 500` ✓

那 4xx 和 5xx 到底差在哪？

4xx — 客戶端的問題

你傳錯了、少帶東西、想看不存在的東西。改前端或改請求就能解決。

5xx — 伺服器的問題

程式炸了、服務掛了。使用者改什麼都沒用，要去修後端。

同一份程式碼，狀態碼是這樣決定的：

```
1  if (方法不對)           → 405 我不接這種請求
2  if (參數不是正整數)     → 400 你送壞了
3  if (找不到這筆商品)    → 404 你送壞了
```



Port 號是什麼

先建立一個畫面

一棟大樓裡有幾百家公司。(find 你的名字 → IP)，接著要找到該走哪一扇門 → Port。

🌐 IP 網址 = 大樓的地址

全網路唯一。 `140.82.121.4`

由 DNS 幫你從域名查出來

🚪 Port 號 = 大樓裡的門牌

一台機器上唯一。 `8080`

由程式啟動時自己指定

技術上：Port 是 16 位元的無號整數

- 範圍 `0 ~ 65535`，共 65536 個
- `0 ~ 1023` 是「保留區」，要系統權限才能用 (`80`、`443`、`22` 都在這)

1 2
3 4

你會一直遇到的 Port 號

3306 MySQL — 資料庫預設連線埠

```
mysql -u root -P 3306
```

5432 PostgreSQL — MySQL 的對手，開源社群常用

```
psql -p 5432
```

6379 Redis — 記憶體快取

```
redis-cli -p 6379
```

27017 MongoDB — 沒有 schema 的資料庫

```
mongod --port 27017
```

8080 HTTP 替代 port — 本課的示範 port

```
http://localhost:8080
```

80 / 443 HTTP / HTTPS — 保留區，要權限才能綁

不用打，直接省略

💡 為什麼 Spring Boot 預設 8080、Express 預設 3000？因為 80/443 要管理員權限，開發時用 1024 以上的 port 最省事

💥 Port 號會撞 — 到底發生什麼事

我們的 Day 2 程式裡這一行：

```
1  HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
2  //                                ↑ 門牌 8080
```

如果 **8080** 已經被別的程式占用（你忘了關掉昨天開的那個），這行直接丟例外，伺服器根本開不起來：

```
1  Exception in thread "main" java.net.BindException:
2    Address already in use
```

(Java 版的錯誤訊息)

怎麼找出誰占了？

```
1  # macOS / Linux
2  lsof -i :8080
3  # 或
4  netstat -an | grep 8080
5  # Windows
6  netstat -ano | findstr :8080
```

```
$ grep -rn "框架" ~/note/
```

什麼是 框架

// 本課最需要慢慢講的一節

① 怎樣才算框架
先有清楚的定義

② 為什麼要有
從你寫過的痛講起

③ 跟 Day2 差在哪
同一件事，兩種寫法



怎樣才算「框架」

先講結論：一句話定義

框架是別人已經幫你打好地基的整套結構，你要在這個結構裡填程式碼，而不是自己從頭蓋。

套件 Library

你呼叫它。控制權在你。

例：`Math.random()`、`new Scanner()`

框架 Framework

它呼叫你。你把程式碼交給它，它決定何時執行。

例：Spring Boot、React、Vue

怎麼分辨？問自己一句話：

「這段程式碼是它主動跑，還是我主動跑？」

我主動跑 → 套件。 它主動跑我寫的 → 框架。



框架幫你蓋了什麼

✗ 沒有框架時，你得手動做這些

- • 自己 new 一個 HTTP 伺服器物件
- • 自己決定綁哪個 port
- • 自己把每個 URL 路徑註冊成處理器
- • 自己讀 request body、解析參數
- • 自己把 Java 物件手刻成 JSON 字串
- • 自己設 Content-Type、自己寫回應串流
- • 自己 try-with-resources 開關資料庫連線
- • 自己記得 close()

✓ 有框架時，這些都不見了

- • 加上 `@RestController`，伺服器自動存在
- • port 寫在設定檔，不是程式碼
- • `@GetMapping("/api/products")` 就是註冊



用程式碼證明：框架省了多少

✗ 原生版

✓ 框架版

NativeApiServer.java – 189 行 / 135 行實體程式碼

ProductController.java – 66 行 / 46 行實體程式碼

```
1 // 綁 port、決定生命週期
2 HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
3 // 手動註冊每一條路由
4 server.createContext("/api/products", NativeApiServer::handleProducts);
5 server.createContext("/api/buy", NativeApiServer::handleBuy);
6 server.createContext("/", NativeApiServer::serveFrontend);
7 server.start(); // ← 忘了這行就不會跑
8
9 // 手刻 JSON：字串拼接 + 跳脫字元
10 StringBuilder json = new StringBuilder("[");
11 json.append("{\"id\":").append(p[0])
12     .append(", \"name\":").append(jsonEscape(p[1]))
13     .append("\", \"price\":").append(p[2]).append('}');
14 // 手動設 header 和狀態碼
15 exchange.getResponseHeaders().set("Content-Type", "application/json; charset=UTF-8");
16 exchange.sendResponseHeaders(200, content.length);
17 try (OutputStream out = exchange.getResponseBody()) { out.write(content); }
```



跟體驗營前兩天有什麼不一樣

面向	Day 1 · 猜數字	Day 2 · 訂單系統	今天 · Spring Boot
程式怎麼開始跑	你寫 <code>main</code> ，你按下去	你寫 <code>main</code> ，你按下去	你還是寫 <code>main</code> ，但一行就完事
資料放哪	變數	SQLite (JDBC)	同上，連線由容器管
別人怎麼找到你	沒人找得到你	<code>localhost:8080</code> + 路徑	同樣的路徑，設定檔改 <code>port</code>
回傳格式	<code>println</code> 到螢幕	手刻 JSON 字串	回傳物件，自動序列化
重複的樣板	很少	很多 (189 行)	幾乎沒有 (66 行)
你學到什麼	Java 語法	HTTP / SQL 怎麼串起來	抽象與分工

🎯 關鍵差異不是「可以少寫多少行」，而是「你現在寫的東西，價值在哪裡」。

Day 2 讓你理解那些笨重是必要的 — 因為不手寫一次，你不會知道框架在解決什麼問題。



什麼時候該用框架

✓ 該用框架

- 要做的是「大家都在做的」那種系統
- 安全性、穩定性比速度重要
- 團隊合作，程式碼要everyone看得懂
- 時間有限，東西要趕快上線

→ 電商、後台系統、API 服務

⚠ 慎用 / 先理解再寫

- 效能吃緊的東西
- 你還不熟這個領域
- 需求可能大幅改變
- 教學 / 練習 / 玩具專案

→ 競賽挑戰、演算法作業、Prototype

⚠ 這句話不要對面試官講

「框架比較好，所以我用框架。」

正確的說法是：「這個問題不需要自己造輪子，我要解決的是 X，不是 Y。」

框架的三大支柱（知道名字就夠了）



約定優於配置

Conventions over Configuration

檔案放對位置、類別取對名字，框架就自動生效。不用寫設定檔。

今天就能看到效果



依賴注入

Dependency Injection

你不用 `new` 依賴的東西，框架送到你手上。這就是 IoC 的實際做法。

`ProductQueryController(seed)`
那種寫法



註解 / 標記

Annotation

用 `@RestController` 告訴框架「這個類別是我的」——宣告意圖，不是執行邏輯。

等於把「註冊」變成一句話



第 10 點你就會親眼看到「約定優於配置」生效：

把 `ProductController` 放對資料夾、加上兩個標記，網址就自動出來了。沒有一行設定檔。



檢查點 2 — 框架

✓ 我能說出

- • 框架 vs 套件的差別
- • 「反向控制」的意思
- • 框架幫我省掉哪幾件事
- • 135 行 → 46 行差在哪

? 我還不太確定

- • 依賴注入到底注入什麼
- • 註解是怎麼被讀到的
- • 為什麼框架要知道我的檔案放哪



馬上複習

回到「 用程式碼證明」那頁，把兩段程式碼並排看一次。



前後端如何溝通

一句話

前端發出**HTTP 請求**，後端回**HTTP 回應**。來回幾個來回就結束。**就這樣，沒有別的了。**



前端

用 `fetch()` 發請求
負責畫面



後端

用 `HttpExchange` 收
負責邏輯



重點：前端「永遠不直接碰資料庫」。它只知道有個網址可以呼叫，資料怎麼存它不管。



一次下單的完整對話





把圖翻成程式碼

① 前端：發請求

```
1  const response = await fetch("/api/products");
2  //           ↑ 圖上的 GET /api/products
3  if (!response.ok) {
4    // ok 為 false 代表狀態碼是 4xx 或 5xx
5    // 200/201 才會是 true
6    throw new Error("讀取失敗");
7  }
8  const products = await response.json();
```

👉 回應對應圖上的「200 · application/json」

② 後端：收請求

```
1  private static void handleProducts(HttpExchange ex) throws IOException {
2    // 圖上的：讀取商品清單
3    sendJson(ex, 200, json.toString());
4  }
```

👉 就是圖上那支箭頭的程式碼



小小體驗 — 三個檔案

我們要跑的三個檔案

```
backend-101-network-framework/  
├─ demo/  
│   ├── NativeApiServer.java    ← 後端 (純 JDK, 零依賴)  
│   └── shop.html               ← 前端 (純 HTML, 零依賴)  
└─ springboot/                 ← 第 10 點才用
```

跟 Day 2 不一樣的地方

- • 沒有 JDBC、沒有 SQLite
- • 不需要 `lib/*.jar`
- • 商品存在 `List` 裡

為什麼？今天要專心看 HTTP 那層，不要被資料庫干擾。

跟 Day 2 一樣的地方

- • 一樣用 `com.sun.net.httpserver`
- • 一樣綁 `8080`
- • 一樣用 `fetch()`

這樣你才看得出框架換掉的是哪一層。



動手 — 啟動原生後端

步驟 1：編譯 + 啟動

```
1 cd demo
2 javac NativeApiServer.java
3 java NativeApiServer
```

看到這幾行就成功了：

```
後端已啟動：http://localhost:8080
瀏覽器前端  http://localhost:8080/
GET  /api/products    讀取商品清單
POST /api/buy          送出訂單
```

步驟 2：先不用瀏覽器，用 curl 看

```
1 # 讀商品
2 curl -i http://localhost:8080/api/products
3
4 # 買 2 杯奶茶
5 curl -i -X POST \
6     -d "productId=1&quantity=2" \
7     http://localhost:8080/api/buy
```

layout: default class: scroll-y



前端要怎麼「借鑑」後端的 API

先理解一件事：API 沒有文件也能用

後端寫好之後，你只要問三個問題，就知道怎麼接：

1 網址跟方法？

直接讀後端程式碼裡的 `createContext("/api/products", ...)` 和 `if (!"GET".equals(...))`

2 要送什麼進去？

看 `form.get("productId")` — key 就是參數名，`parseForm` 決定格式是 form-urlencoded

3 會回什麼？

看 `sendJson(ex, 200, ...)` 裡面那段字串 — 那就是你的 JSON schema

✓ 檢查點 3 — 溝通與實作

✓ 我能說出

- • 前後端溝通就是 HTTP 來回
- • `response.ok` 什麼時候是 false
- • 怎麼從後端程式碼推出 API 規格
- • 為什麼不用 JDBC 也能跑

? 我還不太確定

- • 什麼都沒送會拿到什麼狀態碼
- • 為什麼 form 格式不是 JSON
- • 怎麼知道後端有沒有收到

💡 馬上複習

回到「 一次下單的完整對話」，把圖上的箭頭跟 curl 指令對起來看。

```
$ mvn spring-boot:run
```

配置 魔法

// 同一組 API，換一個寫法

189 行
原生 Java



66 行
Spring Boot

API 完全相同



Spring Boot 怎麼啟動

前提：裝好 JDK 17 以上 + Maven

```
1 java -version    # 需要 17+
2 mvn -version     # 需要 3.8+
```

沒裝 Maven ? [從官網抓 binary 壓縮檔](#) 解壓後把 `bin` 加到 PATH。

啟動

```
1 cd springboot
2
3 # 一行，會先下載依賴再啟動
4 mvn spring-boot:run
```

第一次會比較慢（下載 jar）。看到 `Started Backend101Application` 就是好了。

記得先關掉上一個原生版伺服器

兩個都佔 `8080` → 第二個會直接 `Address already in use`。這就是第 6 點講的撞號。

想打包成 jar 檔再跑？

✨ 魔法在哪 — 整份 Controller

ProductController.java · 66 行

← Day 2 的 BuyProductApi.java 是 189 行

```
1  package com.birc.backend101;                // 套件 → 對應到路徑，框架靠這個找
2
3  @RestController                             // ① 「我是個 API 控制器」
4  @RequestMapping("/api")                     // ② 這個類別的網址都掛在 /api 底下
5  @CrossOrigin(origins = "*")
6  public class ProductController {
7
8      private final List<Product> products = ...; // 資料照舊是記憶體
9
10     @GetMapping("/products")                 // ③ GET /api/products
11     public List<Product> products() {
12         return products;                     // 回傳物件，JSON 自動生成
13     }
14
15     @PostMapping("/buy")                     // ④ POST /api/buy
16     public synchronized ResponseEntity<?> buy(
17         @RequestBody BuyRequest request) {    // ⑤ body 自動綁成物件
18         int id = request.productId();
19         if (id < 1 || id > products.size())
20             return ResponseEntity.badRequest() // ⑥ 一行 = 400
21                 .body(new ErrorResponse("商品與數量必須是正整數"));
```



逐行拆解 — 框架幫你做了什麼

`package` + 資料夾結構

Java 規定：套件名必須對應資料夾。所以 `com.birc.backend101` 必須在 `com/birc/backend101/` — 這是 Java 語言規則，但 Spring 靠它自動掃描。放錯就找不到。

`@RestController`

= 「這個類別的方法，回傳值直接變成 HTTP 回應 body」。取代了手動呼叫 `sendJson`。它 = `@Controller` + `@ResponseBody`。

`@RequestMapping("/api")`

這三個字取代了 `createContext` 註冊。整個類別的網址前綴。

`@GetMapping("/products")`

組成 `GET /api/products`。HTTP 方法 + 路徑，兩個字合併。

`return products;`

回傳 `List<Product>` → 自動序列化成 JSON，自動設 `Content-Type: application/json`，自動呼叫 `sendResponseHeaders`，自動 `close()`。86 行手刻 JSON 變成 1 行。

`@RequestBody`

自動讀 body、解析 JSON、塞進你的物件。取代了 `readAllBytes()` + `parseForm()` + `getOrDefault()`。



公平比較 — 不是 Spring Boot 永遠比較好

如果你要...	用原生 Java	用 Spring Boot
教學、做作業	✓ 看得懂每一步	✗ 黑箱太多
競賽限時 3 小時	✓ 零依賴不會壞	✗ 設定錯誤就開不起來
做真的產品	✗ 你會重寫到哭	✓ 別人都在用
團隊 5 人以上	✗ 樣板碼不一致	✓ 大家寫法統一
需要驗證/ORM/快取	✗ 全部自己刻	✓ 一個 annotation

回到第 7 點的問題：為什麼要有框架？

不是因為框架比較聰明。是因為那些麻煩事（序列化、路由、連線、序列化 JSON）每一個專案都會遇到，而且解法都一樣。把重複的解法抽出來，就是框架。

框架的價值不是「少寫幾行」，是「不用每次都重新想一次」。



檢查點 4 — 全課總複習

網路

- • TCP/IP 四層：線路包話
- • DNS：名字 → IP
- • HTTP：request / response
- • 無狀態的意思

狀態碼

- • 200 成功 / 201 建立了東西
- • 400 請求壞了
- • 401 不知道你是誰
- • 403 知道但你不能
- • 404 不存在
- • 500 伺服器壞了

Port

- • 16 位元，0~65535
- • 唯一性限於同一個 IP
- • 撞號 → BindException
- • 3306 / 8080 / 6379

框架

- • 它呼叫你，不是你呼叫它
- • 反向控制 (IoC)
- • 約定優於配置

溝通

- • 前端不直接碰資料庫
- • response.ok 怎麼用
- • 從程式碼推 API 規格

實作

- • javac → java 跑起來
- • curl 看狀態碼
- • mvn spring-boot:run



下課！

你現在可以解釋網路怎麼跑、狀態碼代表什麼、以及框架到底解決了什麼

這就是後端的真實面貌

回家複習：`localhost:8080` 的 Port 那一頁

下次課程記得先裝好 `Maven`

LucasHsu.dev — 2026 商智中心後端群體驗營